National University of Science and Technology
POLITEHNICA Bucharest
Faculty of Automatic Control and Computers
Computer Science and Engineering Department





DIPLOMA PROJECT

Improving code vulnerability detection using LLMs

Andrei Tudor Fărcășanu

Thesis advisors:

Conf. Traian-Eugen Rebedea

BUCHAREST
2026

Contents

List of figures iii

List of tables iii

Abstract v

1 Introduction 1

2 Related Work 1

2.1 The Challenge of Differentiating Vulnerable and Patched Code 1

2.2 Real-World Vulnerability Detection at Scale 2

2.3 Explainability and Interpretability in Vulnerability Detection 2

2.4 Foundational Deep Learning and Graph-Based Methods 3

3 Data Preparation and Merging 3

3.1 Dataset Version 1: PrimeVul Only 3

3.2 Dataset Version 2: Commit-Level Deduplication 4

3.3 Dataset Version 3: Function-Level Deduplication 4

4 Teacher Model Annotation for Training Data Generation 8

4.1 Teacher Version 1: Plain text prompt 8

4.2 Teacher Version 2: JSON schema and additional hint data 9

4.3 Teacher Version 3: Prompt and Schema Refinements 10

4.4 Teacher Version 4: Hints Re-Introduced for SFT 11

4.5 Teacher Version 5: Commit Message Dropped When CVE Description Present 11

5 Training 11

5.1 Phase 1: Supervised Fine-Tuning (SFT) 11

5.1.1 SFT Version 1: LoRA Fine-Tuning on Qwen3-8B 12

5.1.2 Common Qwen3-4B SFT Configuration (Versions 2+) 12

5.1.3 SFT Version 2: SFT with Hints 13

5.1.4 SFT Version 3: Hint-Free SFT Training Prompts 13
5.1.5 SFT Version 4: SFT with Hints Re-Introduced (ablation) 13
5.1.6 SFT Version 5: SFT with Hints with less commit messages (T5) . . 14
5.2 Phase 2: Group Relative Policy Optimization (GRPO) 14
5.2.1 Reward Function 15
5.2.1.1 GRPO Version 1: 4-Tier Reward Structure 15
5.2.1.2 GRPO Version 2: 5-Tier Reward Structure 15
5.2.1.3 GRPO Version 3: Adjusted 4-Tier Reward Structure with Data Quality Guards 16
5.2.2 GRPO Training Runs 17
5.2.2.1 Run 1: 4-Tier Rewards, Hint-Free SFT Base 17
5.2.2.2 Run 2: 5-Tier Rewards, Hints-in-SFT Base 17
5.2.2.3 Run 3: Adjusted 4-Tier with Data Quality Guards 17

6 Evaluation **19**

7 Results **20**

7.1 Pair-wise Metrics 20
7.2 Combined Dataset Results 20
7.3 PrimeVul Test Set Results 21
7.4 Analysis 21
7.4.1 Overall Progression 22
7.4.2 Initial Experiments: LoRA Fine-Tuning on Qwen3-8B 22
7.4.3 SFT Results 22
7.4.4 GRPO Results 23
7.4.5 Other Observations 23
7.5 Examples 24

8 Conclusion and Future Work **27**

List of Figures

1 Function-level deduplication decision tree 6

2 S1 Training Loss over Steps 12

3 SFT training metrics for all Qwen3-4B models (Tables 1,2): training loss, learning rate, and gradient norm. 14

4 Mean reward across GRPO training steps for all runs 18

5 Mean reward on the validation set during GRPO training for all runs 18

6 Actor gradient norm during GRPO training 19

7 Actor entropy during GRPO training 19

List of Tables

1 Combined Dataset Results – All Model Variants 21

2 PrimeVul Test Results – All Model Variants 21

ABSTRACT

Large Language Models (LLMs) show promise for automated vulnerability detection but struggle to distinguish between vulnerable code and its patched counterpart. This research advances LLM-based vulnerability detection through a multi-stage pipeline combining dataset consolidation, reasoning-augmented fine-tuning, and reinforcement learning. We merge three code vulnerability datasets (PrimeVul, SVEN, SecVulEval) into a new dataset containing 12,912 unique deduplicated functions through a function-level deduplication strategy, then use Qwen3-32B as a teacher model to generate reasoning-augmented training data that teach the student model not just the classification outcome but the analytical process leading to it. Supervised fine-tuning of Qwen3-4B on this dataset, followed by Group Relative Policy Optimization (GRPO) with a tiered reward structure evaluating both classification accuracy and vulnerable line localization, achieves a Pair-wise Correct score of 28.38% on a combined benchmark of 970 vulnerability pairs and 25.93% on the PrimeVul test split, a 100% relative improvement over GPT-4's reported performance in PrimeVul [1]. The GRPO model also surpasses all SFT-only variants, the best of which reaches 25.51% P-C, confirming that reinforcement learning provides a measurable gain beyond supervised fine-tuning alone. Our experiments further reveal that hint-free training prompts produce stronger discriminative models at the cost of output parsability, while iterating on the GRPO reward structure proves critical for controlling undesirable behaviours such as line prediction spraying. Despite these improvements, all configurations still exhibit notable inconsistencies and the overall scores show significant run to run variance, indicating room for improvement.

REZUMAT

Modelele lingvistice de mari dimensiuni (LLM) prezintă potențial pentru detectarea automată a vulnerabilităților în codul sursă, dar întâmpină dificultăți în a distinge între codul vulnerabil și varianta sa corectată. Această lucrare avansează detecția vulnerabilităților în cod sursă folosind LLM-uri printr-un proces în mai multe etape care combină consolidarea a mai multor seturi de date, antrenarea supervizată folosind raționamente generate de un model LLM mai mare și învățarea prin recompensă (reinforcement learning). Îmbinăm trei seturi de date ce conțin vulnerabilități de cod sursă (PrimeVul, SVEN, SecVulEval) în 12.912 funcții unice, pe care le deduplicăm printr-o strategie la nivel de funcție, apoi utilizăm modelul Qwen3-32B ca model profesor pentru a genera un set de date cu modele de raționament care învață modelul student nu doar rezultatul clasificării, ci și procesul analitic care conduce la acesta. Antrenăm supervizat modelul Qwen3-4B pe acest set de date, apoi folosim GRPO cu o structură de recompense pe mai multe niveluri care evaluează atât acuratețea clasificării, cât și localizarea liniilor vulnerabile, obținând un scor Pair-wise Correct (P-C) de 28,38% în benchmark-ul combinat de 970 de perechi de vulnerabilități și 25,93% pe subsetul de test PrimeVul, o îmbunătățire relativă de 100% față de performanța raportată a GPT-4 în lucrarea PrimeVul [1]. Modelul GRPO depășește, de asemenea, toate variantele bazate exclusiv pe antrenarea supervizată, dintre care cea mai bună atinge 25,51% P-C, confirmând faptul că învățarea prin recompensă oferă un câștig măsurabil dincolo de antrenarea supervizată. Experimentele

noastre arată că prompturile de antrenament fără indicii (hint-free) produc modele cu o capacitate discriminativă mai puternică, dar cu un cost asupra parsabilității ieșirilor, în timp ce structura de recompense folosită la GRPO se dovedește esențială pentru controlul comportamentelor nedorite, cum ar fi predicția excesivă a liniilor. În ciuda acestor îmbunătățiri, toate configurațiile prezintă instabilitate, iar scorurile generale arată o variantă semnificativă între rulări, indicând faptul că sunt posibile îmbunătățiri suplimentare.

1 Introduction

The escalating complexity of modern software systems has been paralleled by a surge in security vulnerabilities, making automated detection a critical field of research. While traditional static and dynamic analysis tools have been foundational, they often struggle with high false positive rates or require extensive manual configuration. The advent of Large Language Models (LLMs) has introduced a new paradigm, demonstrating a remarkable ability to understand and generate code, which has naturally been extended to the task of vulnerability detection [2, 3].

Despite their perceived potential, more recent works have shown that LLMs often fail to distinguish between vulnerable code and its subtly patched, non-vulnerable counterpart. This limitation, highlighted in works like PrimeVul [1], suggests that models may be overfitting to superficial textual features rather than grasping the deep, underlying semantic causes of a vulnerability. When presented with a function and its nearly identical fixed version, many state-of-the-art models struggle to label both correctly, raising serious concerns about their reliability for practical security applications.

Inspired by the knowledge-level enhancement frameworks proposed by works like Vul-RAG [4], we hypothesized that an LLM's ability to differentiate vulnerable from benign code can be significantly improved by fine-tuning it on a dataset that explicitly articulates the reasoning behind a vulnerability's existence and its corresponding fix. This research approaches this challenge through a multi-stage methodology: establishing baseline performance of modern open-source LLMs on the PrimeVul benchmark; generating a reasoning-augmented fine-tuning dataset by prompting a more powerful teacher model to articulate its analysis; scaling this approach across multiple vulnerability datasets while minimizing redundancy across complementary data sources; and applying reinforcement learning with targeted reward signals to further optimize the model's discriminative capabilities. The expected outcome is a vulnerability detection system that achieves superior discrimination between vulnerable and patched code, scales effectively to larger and more diverse datasets, and generalizes better to vulnerability patterns.

2 Related Work

2.1 The Challenge of Differentiating Vulnerable and Patched Code

Recent works highlight the difficulty LLMs face in distinguishing between vulnerable code and its minimally altered, patched version. The PrimeVul paper [1] highlighted this issue by introducing a benchmark composed of paired vulnerable and fixed code functions and proposed a set of pair-wise evaluation metrics (P-C, P-V, P-B, P-R) to specifically measure this discriminative capability. Their findings revealed that even powerful models such as GPT-4 struggled to correctly classify both samples in a vulnerable/benign pair, with P-C scores

indicating significant room for improvement. This observation suggests that LLMs may be overfitting to superficial textual features rather than grasping the deep, underlying semantic causes of vulnerabilities.

2.2 Real-World Vulnerability Detection at Scale

The CyberGym benchmark [5] demonstrates the complexity of real-world vulnerability detection through a large-scale evaluation framework featuring 1,507 vulnerabilities across 188 open-source projects discovered by OSS-Fuzz. Rather than simple classification tasks, CyberGym requires AI agents to generate proof-of-concept tests that reproduce vulnerabilities given only vulnerability descriptions and source code repositories containing thousands of files and millions of lines of code. The benchmark reveals that even the top-performing AI agent combinations (OpenHands with Claude-3.7-Sonnet) achieve only 11.9% success rate on the primary task, underscoring the substantial gap between current capabilities and practical vulnerability analysis. Beyond benchmarking, CyberGym demonstrates tangible real-world impact: agents discovered 35 zero-day vulnerabilities and identified 17 historically incomplete patches in production software.

Building on the challenge of repository-scale analysis, Just-In-Time (JIT) vulnerability detection has emerged as another problem because vulnerabilities often manifest through multi-hop function calls across repositories rather than within isolated functions. The JitVul benchmark [6] addresses this limitation by linking each vulnerable function to its vulnerability-introducing and fixing commits across 879 CVEs spanning 91 vulnerability types. JitVul demonstrates that LLM-based agents with reasoning capabilities outperform isolated function-level analysis, with ReAct agents leveraging thought-action-observation patterns achieving superior discrimination between vulnerable and benign code compared to standard LLM prompting.

2.3 Explainability and Interpretability in Vulnerability Detection

Understanding why models make vulnerability predictions is increasingly important for both security practitioners and model developers. Recent work on Explainable Vulnerability Detection [7] using edge-aware graph attention networks demonstrates how to compute relevance scores for both nodes and edges in Code Property Graphs by leveraging attention weights and input gradients, then mapping these back to source-level constructs for human interpretation. This approach enables security teams to understand which specific code elements the model identified as problematic, bridging the gap between black-box model predictions and actionable insights.

2.4 Foundational Deep Learning and Graph-Based Methods

While LLM-based approaches represent the current frontier, foundational deep learning methods remain important references. Graph Neural Networks operating on Abstract Syntax Trees and Control Flow Graphs [8] have demonstrated effective vulnerability detection by capturing semantic program structure. Deep learning systems like VulDeePecker [9] and SySeVR [10] pioneered neural approaches to vulnerability identification. More recent dataflow-inspired approaches [11] show that combining traditional program analysis insights with deep learning continues to yield improvements. These foundational techniques inform our understanding of what semantic patterns are critical for vulnerability detection and suggest that enhancing LLMs with structured reasoning and rich training signals is a promising direction.

3 Data Preparation and Merging

The dataset preparation evolved across a couple of major versions to maximize data diversity while ensuring accurate pair-wise evaluation. The pipeline combines three primary data sources:

PrimeVul serves as the primary foundation, providing paired vulnerable and patched code functions from CVEs. The dataset includes commit-level metadata linking each vulnerability to its fix, and vulnerability descriptions.

SVEN contributes 1,606 manually-verified functions (803 vulnerable, 803 non-vulnerable) covering 9 CWE types with claimed 94% label accuracy. SVEN's distinguishing feature is its detailed metadata including the specific lines that were changed between vulnerable and patched versions as well as character level changes.

SecVulEval adds additional CVE and CWE linked samples, with line level change metadata.

3.1 Dataset Version 1: PrimeVul Only

In our initial experiments, we relied exclusively on the paired dataset provided by the PrimeVul benchmark [1]. This dataset consists of paired vulnerable functions and their corresponding fixed, non-vulnerable versions, providing 1,632 entries (816 pairs), which were used to establish baseline performance and conduct early dataset generation experiments. However, relying solely on the stock PrimeVul dataset limited the diversity and metadata richness of our training data. This limitation prompted the integration of additional datasets and deduplication strategies in subsequent versions to support full-scale fine-tuning and reinforcement learning.

3.2 Dataset Version 2: Commit-Level Deduplication

The initial merging process followed a priority-based deduplication strategy using `commit_id` as the primary key. When samples from multiple sources shared the same commit, SVEN data was prioritized due to its manual verification, followed by merging CVE descriptions from any available source. The merging script preserves all `line_changes` information, which is critical for training the model to identify specific vulnerable lines.

```
{
  "source": "primevul",
  "commit_id": "239c4f7...",
  "project": "wget",
  "cve": "CVE-2015-7665",
  "cve_desc": "Tails before 1.7 includes the wget program but
     does not prevent automatic fallback from passive FTP...",
  "vuln_func": "void function(...) { ... }",
  "patched_func": "void function(...) { ... }",
  "deleted_lines": [
    {"line_no": 645, "text": "err = ftp_do_port (csock, ...);"}
  ],
  "added_lines": []
}
```

Listing 1: Example of a merged dataset entry (structure)

After commit-level deduplication and filtering out samples where the vulnerable function was identical to the patched function, this consolidated dataset contained approximately 6,600 unique samples.

3.3 Dataset Version 3: Function-Level Deduplication

While commit-level deduplication prevented identical overlaps, it introduced a significant flaw: commits that patched multiple different functions were collapsed into a single pair, dropping thousands of valid vulnerability pairs.

To address this, we implemented a function-level deduplication pipeline. The process groups samples by `(commit_id, func_name)` and applies a three-tier conflict resolution strategy. First, each function's code is normalized (whitespace stripped, empty lines removed, lower-cased) and assigned a SHA-256 hash. Entries within the same group that share an identical hash are true duplicates; the best entry is selected by source priority (SVEN $>$ SecVulEval $>$ PrimeVul) and metadata from all losers is merged into the winner (fill-only semantics: existing fields are never overwritten).

When entries in the same group have different hashes, a pairwise resolution determines whether they represent genuine distinct functions or merely crawling artifacts from different sources. For each pair, we compute a normalized diff to determine how many lines were added and

removed, a `SequenceMatcher` similarity score, and whether the functions originate from different files. If one function wholly contains the other (i.e., no lines were removed, only added) or the pair exhibits high similarity (> 0.7) or a high added-lines ratio (≥ 2), and both entries come from the same file path, they are collapsed: the version with more added lines is kept, with tie-breaking by CVE description availability, commit message availability, and finally source priority. If the pair has low similarity *and* a low added-lines ratio, or if the entries come from different files, they are treated as genuinely different functions and both are preserved. After resolution, all surviving variants within a group undergo cross-enrichment: metadata gaps such as missing CVE and CWE identifiers are filled from siblings, while missing CVE descriptions are only imported if the variants' CVE identifiers are compatible (matching or subset).

To stabilize downstream pairwise evaluation and enable the hint-free prompt regeneration introduced in T3, we also introduced deterministic `pair_ids` by appending the `vuln_hash` to each sample's identifier, ensuring that pairs remain uniquely trackable across evaluation runs. The conflict resolution logic is summarized in figure 1:

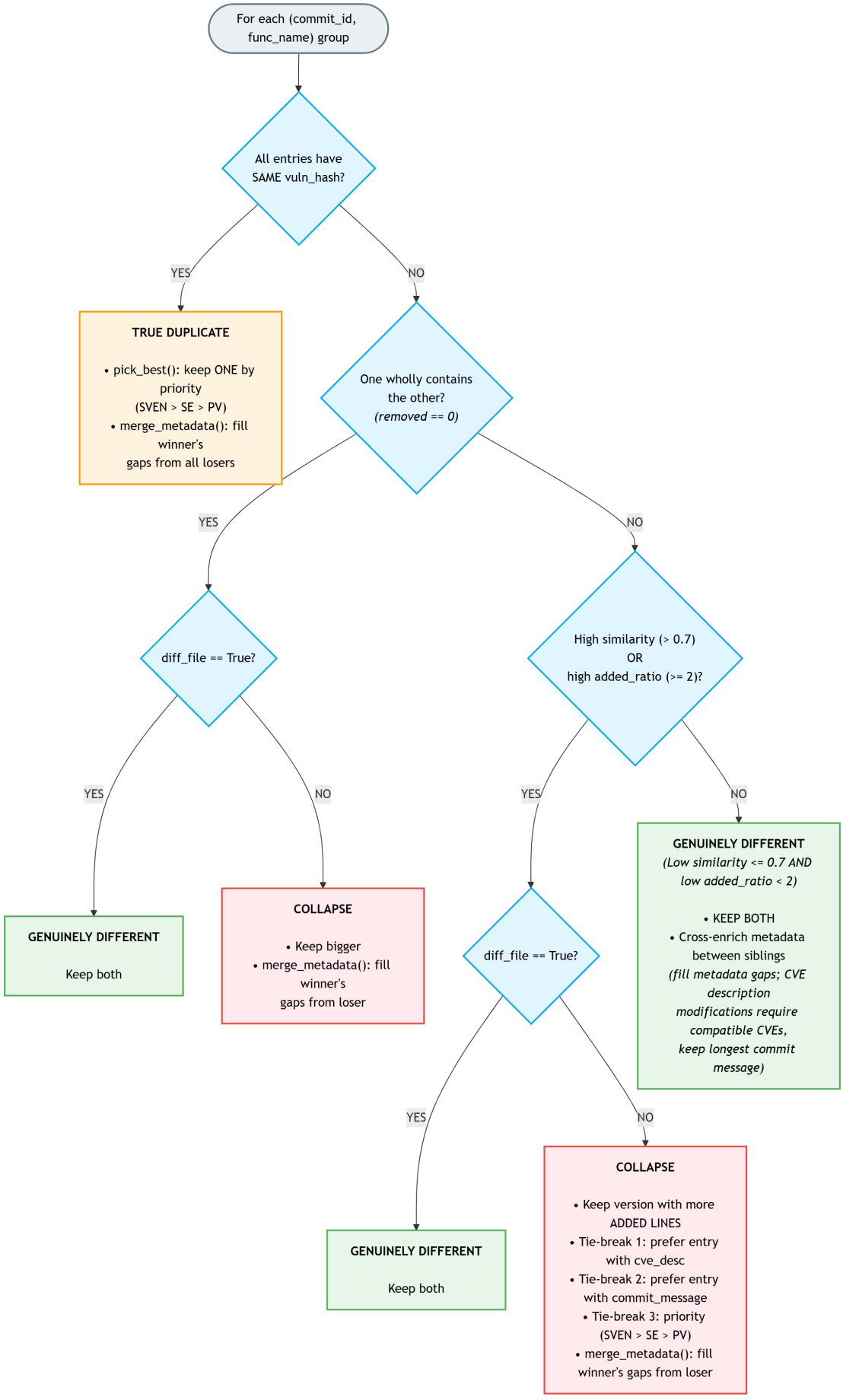


Figure 1: Function-level deduplication decision tree

As part of the dataset refinement, we also implemented several data quality measures. Ground truth line numbers from SecVulEval were found to contain out-of-bounds indices and misindexed lines, we corrected these by first attempting text-based search to relocate the correct line, falling back to a difflib approach computing the exact line changes between the vulnerable and patched functions directly. Samples where the vulnerable function was effectively identical to its patched counterpart (exact match or only trailing whitespace differences) were filtered out, as they represent non-fixes. Additionally, uninformative CWE classifications such as "NVD-CWE-noinfo" and "NVD-CWE-Other" were excluded from the metadata to avoid noise in downstream analysis.

This function-level deduplication and data quality pipeline successfully recovered previously discarded data, expanding the final consolidated dataset to 12,912 unique deduplicated functions.


```
{
  "source": "primevul",
  "pair_id": "primevul_wget_239c4f7_ftp_do_port_bf051808...",
  "commit_id": "239c4f7...",
  "project": "wget",
  "project_url": "https://github.com/bminor/wget",
  "func_name": "ftp_do_port",
  "filepath": "src/ftp.c",
  "vuln_hash": "bf05180879eea857f1998742f452dfb7e7fa1fbc10...",
  "vuln_func": "void ftp_do_port(...) { ... }",
  "patched_func": "void ftp_do_port(...) { ... }",
  "cve": ["CVE-2015-7665"],
  "cve_desc": "Tails before 1.7 includes the wget program but
     does not prevent automatic fallback from passive FTP...",
  "cwe": ["CWE-200"],
  "commit_message": "Fix FTP fallback information disclosure...",
  "deleted_lines": [
    {
      "line_no": 645,
      "char_start": 1032,
      "char_end": 1063,
      "text": "err = ftp_do_port (csock, ...);"
    }
  ],
  "added_lines": []
}
```


Listing 2: Example of the function-level deduplicated dataset entry structure

4 Teacher Model Annotation for Training Data Generation

To create training data for the reinforcement learning phase, we implemented a teacher model annotation pipeline using Qwen3-32B. We generated reasoning traces for each vulnerability sample, following structured prompt formats designed to elicit both classification and localization information.

4.1 Teacher Version 1: Plain text prompt

In our initial experiments, the prompting strategy was designed to elicit a simple, structured classification response without relying on a formal JSON schema. Each prompt contained the full, numbered code snippet, the associated CVE description if available, and ground-truth hints explicitly stating whether the snippet was vulnerable and pointing to the modified lines, and finally, a direct instruction forced the model to perform its reasoning internally before providing a parseable answer: "Please indicate your analysis result *only* with one of the following exact statements...: '(1) YES:...' or '(2) NO:...'".

```
Please analyze the following code snippet:
```
...
774:            if (!opt.server_response)
775:              logputs (LOG_VERBOSE, "==> RETR ... ");
...
798:        err = ftp_retr (csock, u->file);
...
800:        switch (err)
801:          {
802:          case FTPRERR:
...
904:       }
```

The associated CVE description is: Tails before 1.7 includes the wget ...
Hint: According to the ground truth, this code snippet is considered vulnerable.
The changed lines that might be related to the vulnerability are: Lines : 11, 642, 645, 646, 648.

Please indicate your analysis result with one of the options:
(1) YES: A security vulnerability detected.
(2) NO: No security vulnerability.

Only reply with one of the options above. Do not include any further information.
```

Listing 3: Example teacher annotation prompt from Semester 2 (using the wget example)

4.2 Teacher Version 2: JSON schema and additional hint data

```
You are an expert code security analyst. Analyze the provided
code and determine if it contains any security vulnerabilities.

Code:
...
774:            if (!opt.server_response)
775:              logputs (LOG_VERBOSE, "==> RETR ... ");
...
798:        err = ftp_retr (csock, u->file);
...
800:        switch (err)
801:          {
802:          case FTPRERR:
...
904:       }

Context:
- CVE: CVE-2015-7665
- Description: Tails before 1.7 includes the wget program but
  does not prevent automatic fallback from passive FTP to active
  FTP, which allows remote FTP servers to discover the Tor client
  IP address.

Hint: This code contains a security vulnerability.
The changed lines that might be related to the vulnerability are:
Lines: 11, 642, 645, 646, 648.

Task: After your reasoning, provide your analysis in JSON format:
{"classification": "VULNERABLE" or "NOT_VULNERABLE",
 "vulnerable_lines": [...], "reasoning_summary": "..."}
```

Listing 4: Example system prompt (truncated) used for teacher model annotation

Each sample continues to be formatted with line numbers to facilitate vulnerable line identification, but this time the prompt includes additional contextual hints derived from the merged dataset: CVE identifiers, CWE classifications, CVE descriptions (where available), and the specific lines that were changed. For vulnerable samples, the deleted lines between vulnerable and patched versions serve as ground truth for the `vulnerable_lines` field (later renamed to `important_lines` to unify the schema across vulnerable and patched samples).

The annotation process prioritizes samples with rich metadata (CVE descriptions). The pipeline uses SGLang [12] for inference with Qwen3-32B, with concurrent requests.

4.3 Teacher Version 3: Prompt and Schema Refinements

This version introduced several refinements to the teacher annotation pipeline. The prompt construction was updated to dynamically include commit messages when token budget allows, with intelligent truncation: if a sample has a CVE description, the commit message is dropped when it would push the prompt below a minimum output size threshold, while if no CVE description is available, the commit message is token-level truncated to fit the remaining space.

Additionally, the output field `vulnerable_lines` was renamed to `important_lines` so that the output schema makes sense across vulnerable and patched samples, enabling the GRPO reward function to evaluate line localization for patched (fixed) code using `added_lines` as ground truth.

For the same sample shown above, the T3 teacher annotation prompt would appear as follows, including the added commit message and updated JSON schema:

```
You are an expert code security analyst. Analyze the provided
code and determine if it contains any security vulnerabilities.

Code:
...
774:            if (!opt.server_response)
775:              logputs (LOG_VERBOSE, "==> RETR ... ");
...
798:        err = ftp_retr (csock, u->file);
...
904:      }

Context:
- CVE: CVE-2015-7665
- Description: Tails before 1.7 includes the wget program but
  does not prevent automatic fallback from passive FTP to active
  FTP, which allows remote FTP servers to discover the Tor client
  IP address.
- Commit Message: Fix passive FTP fallback vulnerability.

Hint: This code contains a security vulnerability.
The changed lines that might be related to the vulnerability are:
Lines: 11, 642, 645, 646, 648.

Task: After your reasoning, provide your analysis in JSON format:
{"classification": "VULNERABLE" or "NOT_VULNERABLE",
 "important_lines": [...], "reasoning_summary": "..."}
```

Listing 5: Example T3 teacher annotation prompt

4.4 Teacher Version 4: Hints Re-Introduced for SFT

This version is functionally identical to T3 for the purpose of teacher annotation. The distinction is in how the resulting dataset is consumed downstream: whereas T3 was paired with hint-free SFT prompts (S3), this version re-introduces the full hint-rich prompt during the SFT training phase as well (S4), while GRPO continues to use hint-free prompts exclusively.

4.5 Teacher Version 5: Commit Message Dropped When CVE Description Present

Upon analysing the reasoning traces produced by the teacher model under T4 (and by extension T3), we observed that in a significant fraction of cases the model devoted roughly half of its reasoning trace to discussing the metadata hints (CVE description, commit message) rather than performing code-level analysis. The commit message in particular was often given much reasoning, possibly because it was often much longer than the CVE description, while also usually containing potentially irrelevant information such as email addresses of the authors of the patch commits, adding little independent signal while consuming a substantial portion of the model's reasoning budget.

To address this redundancy, T5 modifies the prompt construction to unconditionally drop the commit message whenever the sample already has a CVE description. Samples lacking a CVE description continue to receive the commit message (token-truncated to fit the remaining context budget), since it may be the only available natural-language context about the vulnerability. This change preserves the metadata richness of the prompt while reducing reasoning dilution, encouraging the teacher model to focus more on the code itself.

5 Training

5.1 Phase 1: Supervised Fine-Tuning (SFT)

In phase 1 we trained Qwen3-4B, and initially Qwen3-8B, to mimic the teacher model's responses. This phase uses the generated dataset where each sample contains the formatted prompt (in the various ways also described in 4) and the teacher's response (including full reasoning trace in "thinking mode"). The model learns to produce structured responses, at first formatted just like the "`std_cls`" prompt format from PrimeVul[1], then later on JSON outputs with classification, vulnerable line identification, and reasoning summaries.

5.1.1 SFT Version 1: LoRA Fine-Tuning on Qwen3-8B

Using the initial PrimeVul-only dataset annotated with the basic classification prompt (T1), we performed an initial round of Low-Rank Adaptation (LoRA) fine-tuning on a 4-bit quantized Qwen3-8B model using Unsloth [13], in order to test whether teaching a model the explicit reasoning behind vulnerabilities could improve its discriminative capabilities before scaling up to full parameter fine-tuning.

To make training computationally feasible on a single 80GB GPU, we used **Low-Rank Adaptation (LoRA)**, a parameter-efficient fine-tuning (PEFT) technique that freezes the pre-trained model weights and injects trainable rank-decomposition matrices into the layers of the Transformer architecture, significantly reducing the number of trainable parameters. We applied LoRA adapters across all attention projections (`q_proj, k_proj, v_proj, o_proj`) and feed-forward layers (`gate_proj, up_proj, down_proj`). We utilized an expressive configuration with a LoRA Rank (r) of 128, which determines the rank of the decomposition matrices ($W_\Delta = B \cdot A$). While a higher rank increases the parameter count, it also allows for more expressive power in the adapter, so we chose a somewhat higher value to ensure the adapter had sufficient expressive power for significant adaptation. This was paired with a LoRA Alpha (α) of 128, a scaling factor for the LoRA output that controls the magnitude of the adaptation relative to the original weights, which is typically set equal to the rank. Additional training hyperparameters included a learning rate of 2×10^{-4} with a linear scheduler, an effective batch size of 32 (4 per device $\times$ 8 gradient accumulation steps), and a training duration of one full epoch. As shown in Figure 2, the training loss exhibited a stable decrease, confirming effective learning from the reasoning-augmented dataset (T1).

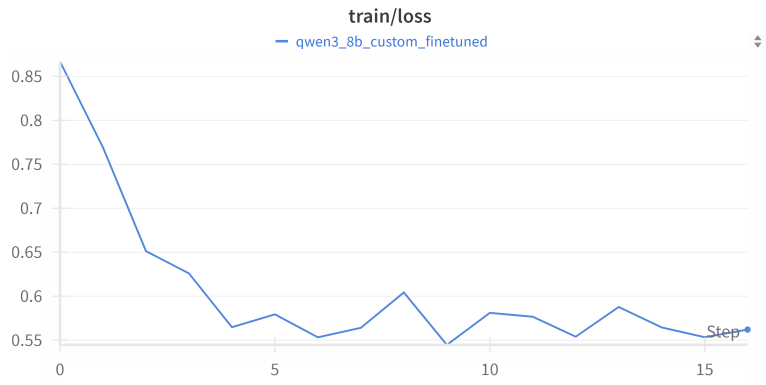


Figure 2: S1 Training Loss over Steps

5.1.2 Common Qwen3-4B SFT Configuration (Versions 2+)

Building upon the success of the initial LoRA experiment, all subsequent SFT iterations transitioned to full parameter fine-tuning of Qwen3-4B. This model was selected as the primary training target because it possesses native "thinking mode" capabilities for detailed reason-

ing traces, yet fits within the 3×80GB GPU memory constraint for full fine-tuning using DeepSpeed ZeRO-3 [14]. All Qwen3-4B SFT runs were executed on the UPB cluster (fep) utilizing 3×80GB H100 GPUs, 64 CPU cores, and 300GB RAM, with the following standard hyperparameters:

- Learning rate: 2×10^{-5} with 3% warmup
- Batch size: 1 per device × 16 gradient accumulation steps
- Max sequence length: 32,768 tokens
- 1 epoch, gradient checkpointing enabled

5.1.3 SFT Version 2: SFT with Hints

This version represents the first full-parameter fine-tuning run on Qwen3-4B, trained on the commit-level deduplicated dataset (D2) with teacher annotations from T2. Both prompts and responses included vulnerability hints and metadata context during training.

5.1.4 SFT Version 3: Hint-Free SFT Training Prompts

While the teacher model annotation uses the full hint-rich prompt to generate high-quality reasoning traces, in this version for the SFT training dataset we reconstruct hint-free prompts from the merged dataset so that the student model receives only the raw code with line numbers and the JSON output instruction, without any CVE context or vulnerability hints, in hope of making the SFT model more used to the evaluation prompts, while still learning from the teacher's response that was generated with full context. This change compared to the previous version S2 aims to make the SFT model more robust and improve its ability to identify vulnerabilities from the code itself rather than relying on metadata cues.

However, this version started to exhibit low parseability, where during several evaluation runs, at all the recorded temperatures (0.2, 0.4, 0.6) there was a noticeable drop (about ∼200-300 less) in parsable responses due to the model getting stuck in the reasoning phase, when compared to the previous version S2.

5.1.5 SFT Version 4: SFT with Hints Re-Introduced (ablation)

Due to the perceived reduction in parsable responses of S3, as an ablation study, we re-introduced the full hint-rich prompt during the SFT phase using the T4 teacher annotations. The hypothesis was that providing vulnerability context again (CVE descriptions, changed lines) inside the prompts (not only in the target responses) during supervised learning, just like in S2, might help the model develop more stable reasoning patterns, even though GRPO evaluation would still use hint-free prompts. This version used the function-level deduplicated dataset (D3).

This had the desired effect, as can be seen in table 1, where the parsability of S4 models is almost back to the level of S2.

5.1.6 SFT Version 5: SFT with Hints with less commit messages (T5)

This version pairs the SFT training with the refined teacher annotations from T5, which drop the commit message when a CVE description is already present. The motivation was that traces from the previous teacher version showed the model spending a disproportionate amount of reasoning on paraphrasing the commit message rather than analysing the code, potentially degrading the quality of the SFT learning signal.



Figure 3: SFT training metrics for all Qwen3-4B models (Tables 1,2): training loss, learning rate, and gradient norm.

Figure 3 shows that all four SFT runs converge to a comparable final training loss ($\sim$0.55–0.60), with the S2 run (trained on the smaller commit-level dataset) converging fastest due to fewer training samples. The learning rate schedules differ slightly across runs, reflecting different dataset sizes and warmup ratios, but the gradient norms remain relatively stable throughout training for all variants, indicating that full-parameter fine-tuning on this task does not suffer from gradient instability.

5.2 Phase 2: Group Relative Policy Optimization (GRPO)

Phase 2 refines the SFT checkpoint using reinforcement learning with a programmatic reward function. GRPO, introduced in the DeepSeekMath paper [15], is a variant of Proximal Policy Optimization (PPO) specifically designed for language model training.

The key innovation of GRPO is eliminating the need for a separate value network (critic), reducing memory overhead by 40–60%. Instead of learning a value function, GRPO estimates the baseline for advantage calculation directly from the rewards of multiple outputs generated for the same prompt. For each prompt q, the model generates G candidate responses $\{o_1, ..., o_G\}$, each receiving a reward r_i. The advantage for each response is computed as:

$$A_i = \frac{r_i - \text{mean}(\{r_1, ..., r_G\})}{\text{std}(\{r_1, ..., r_G\})} \tag{1}$$

This group-relative advantage indicates whether a response is better ($A_i > 0$) or worse ($A_i < 0$) than the group average. The policy is then updated using the standard PPO clipped objective with these advantages [15, 16].

5.2.1 Reward Function

The reward function implements a discrete structure based on classification accuracy and vulnerable line localization. Over the course of our experiments, we evaluated these configurations:

5.2.1.1 GRPO Version 1: 4-Tier Reward Structure

This initial reward configuration scored both vulnerable and patched samples for line localization: vulnerable samples were evaluated against their ground truth `deleted_lines`, while patched samples were scored against their `added_lines`. The rollout temperature was set to 1.0 for maximum exploration.

- **Full reward (1.0):** Correct classification and $\geq$50% of important lines correctly identified
- **Partial reward (0.6):** Correct classification but insufficient line localization ($<$50%)
- **Localization-only reward (0.3):** Incorrect classification but some correct important lines identified
- **Format reward (0.05):** Valid JSON output but wrong classification and no correct lines
- **Zero reward (0.0):** Invalid response format (malformed JSON, incomplete output)

5.2.1.2 GRPO Version 2: 5-Tier Reward Structure

To reduce the number of unparsable responses (where the model failed to output valid JSON) and reward high-precision localization, we tested an expanded 5-tier system. Like G1, this version scored both vulnerable and patched samples for line localization. The rollout temperature was reduced to 0.975. Additionally, the SFT checkpoint (S4) used for this run was trained with hints re-introduced in the prompts (T4), in order to see if not using hints in the prompts during SFT caused the increase of the number of unparsable responses in D3-T3-S3-G1 and D3-T3-S3, compared to D2-T2-S2.

- **High-Precision reward (1.25):** Correct classification and $\geq$75% of important lines identified
- **Full reward (1.0):** Correct classification and $\geq$50% of important lines identified
- **Partial reward (0.75):** Correct classification

- **Localization-only reward (0.4):** Incorrect classification but some correct lines identified
- **Format reward (0.15):** Valid JSON output
- **Zero reward (0.0):** Invalid response format (malformed JSON, incomplete output)

5.2.1.3 GRPO Version 3: Adjusted 4-Tier Reward Structure with Data Quality Guards

Analysis of training rollouts from the G2 run revealed that the model had learned to "spray" a large number of predicted lines, particularly on patched samples where the ground truth `added_lines` are logically uninferable from the code alone. The 5-tier structure, with its 1.25 high-precision tier, inadvertently incentivised this behaviour by rewarding models that guessed many lines (increasing the chance of hitting $\geq$75% overlap). Furthermore, samples with an unusually large number of ground truth lines (e.g., 100+ changed lines, representing extensive refactoring rather than targeted security patches) introduced noise into the line localization signal.

To address these issues, G3 returns to a 4-tier reward structure with adjusted values and introduces two data quality guards:

- **Full reward (1.0):** Correct classification and $\geq$50% of important lines correctly identified
- **Partial reward (0.75):** Correct classification only
- **Localization-only reward (0.3):** Incorrect classification but some correct lines identified
- **Format reward (0.1):** Valid JSON output but wrong classification and no correct lines
- **Zero reward (0.0):** Invalid response format

The **GT line capping** guard zeroes out the ground truth lines for training samples whose ground truth line count exceeds the 99th percentile (which was computed over vulnerable samples only, as for this specific training run the patched sample scoring was disabled, with the threshold being approximately 46 lines after applying a 1.15 multiplier to the figure the 99th percentile returned), affecting roughly 0.3% of the dataset. These samples still contribute their classification signal but no longer provide a noisy line localization reward. The **spray guard** is an optional runtime mechanism (which in the case of this run was set to a somewhat lenient 60 lines, so as to not discourage the model in the case of legitimate but very long fixes) that withholds the line localization bonus when the model predicts more lines than a configurable threshold, giving only the classification-level reward. Finally, patched sample scoring was disabled for this version: during GRPO the reward function evaluates line localization only on vulnerable samples, since the model cannot logically deduce which lines were added to a patch from the patched code alone.

5.2.2 GRPO Training Runs

All GRPO runs shared the following common configuration: learning rate 1×10^{-6}, training batch size of 60 prompts per iteration, mini-batch size 48, micro-batch size 1 per GPU, max context of 40,960 tokens (28,680 prompt + 12,279 response), FSDP2 with parameter and optimizer offload across 3 H100 GPUs, and KL coefficient $\beta = 0$ following recent research showing that KL penalties are not essential for GRPO-style training [17, 18]. The rollout backend was SGLang [12] with Top-P 0.95 and Top-K 20, and all runs used $G = 16$ generations per prompt. The full deduplicated dataset (D3, 24,440 training samples, 1,286 validation samples) was used for all runs. The three runs mainly differed in their SFT starting checkpoint, reward version, and rollout temperature.

5.2.2.1 Run 1: 4-Tier Rewards, Hint-Free SFT Base

This run used the full deduplicated dataset (D3, 24,440 training samples, 1,286 validation samples) with hint-free prompts for both SFT and GRPO. Starting from the S3 checkpoint, the G1 reward structure was used with rollout temperature 1.0.

5.2.2.2 Run 2: 5-Tier Rewards, Hints-in-SFT Base

Starting from the S4 checkpoint (where hints were re-introduced during SFT), this run tested the G2 reward structure with a slightly reduced rollout temperature of 0.975.

5.2.2.3 Run 3: Adjusted 4-Tier with Data Quality Guards

Starting from the S5 checkpoint, this run used the G3 reward structure with the spray guard and GT line capping enabled. Patched sample scoring was disabled, and the rollout temperature was set to 0.975. This run was evaluated at multiple checkpoints (step 180 and step 407) to assess training dynamics.

Figures 4 and 5 show the training dynamics across all three runs. The mean reward (Figure 4) reveals that Run 2 (G2) achieves the highest absolute reward ($\sim$0.55–0.65), which is expected since its 5-tier structure has more lenient thresholds and a higher maximum tier (1.25). Run 3 (G3) settles at $\sim$0.45–0.55, while Run 1 (G1) hovers around $\sim$0.40–0.45. All three runs exhibit somewhat substantial step-to-step oscillation, possibly because each batch presents a different mix of easy and hard samples, or the high temperature promotes significant exploration.

The validation accuracy during training (Figure 5) shows a similar picture, with Run 2 reflecting the overall higher rewards used, while the best P-C scoring Run 1 has the lowest reward and as such showing lower on the graphic, highlighting the need for the pair-wise evaluation methodology.

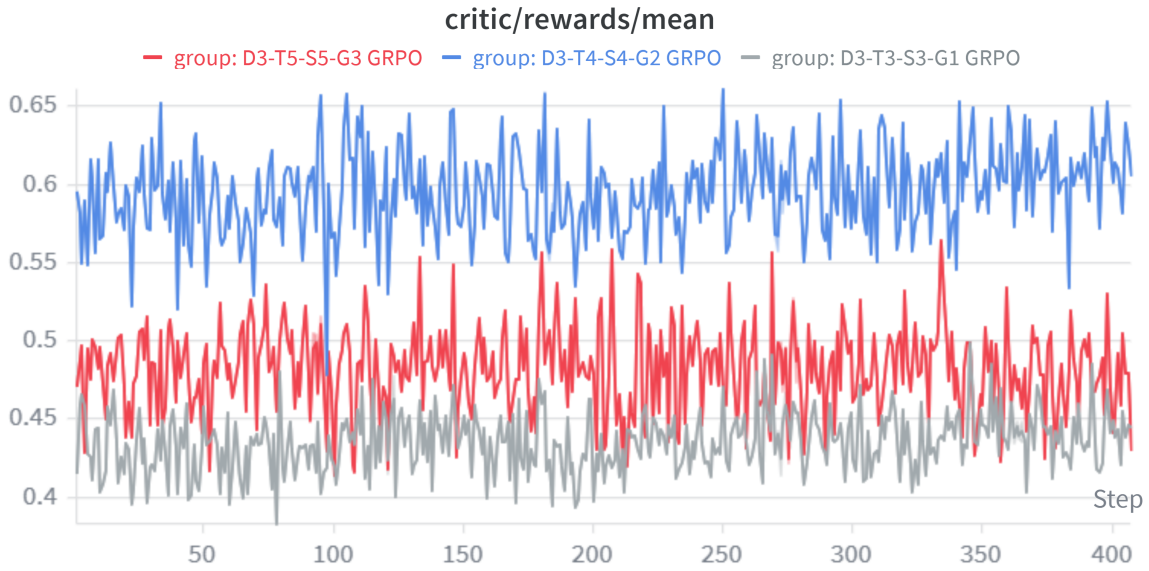


Figure 4: Mean reward across GRPO training steps for all runs

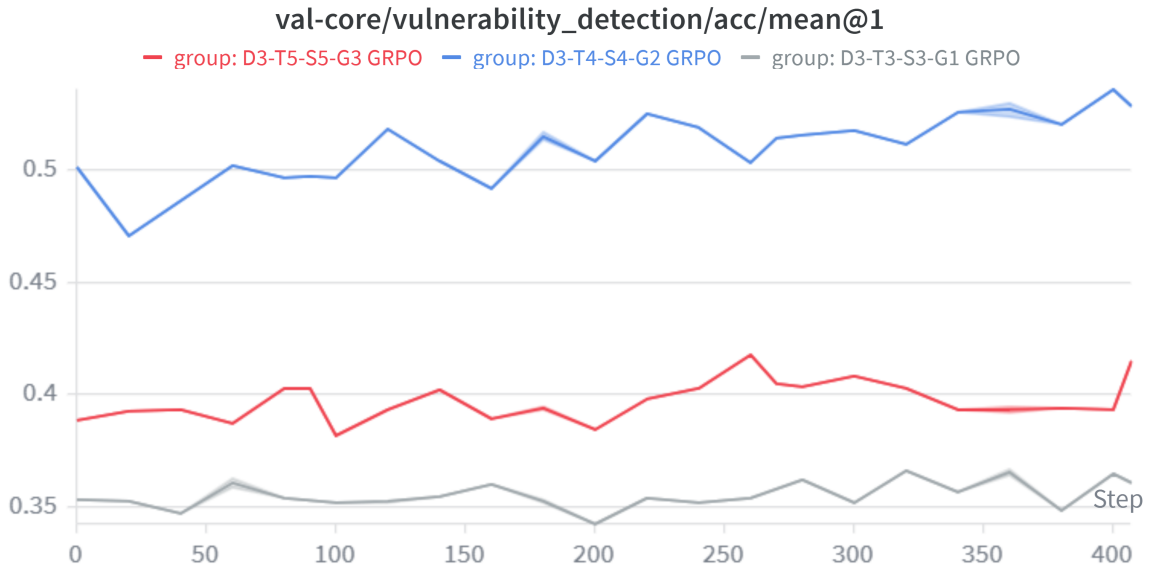


Figure 5: Mean reward on the validation set during GRPO training for all runs

The actor gradient norm and entropy curves (Figures 6 and 7) show that the training was relatively stable. Gradient norms remain low across all runs (below 0.2 on average, with only rare isolated spikes), indicating no gradient instability or divergence, and the entropy remains relatively flat throughout training for all three runs, with no signs of entropy collapse. Run 1 (D3-T3-S3-G1) exhibits slightly lower entropy ($\sim$0.57) compared to Runs 2 (D3-T4-S4-G2) and 3 (D3-T5-S5-G3, $\sim$0.60), consistent with the hint-free SFT being less explorative than the others, which were trained with hints.

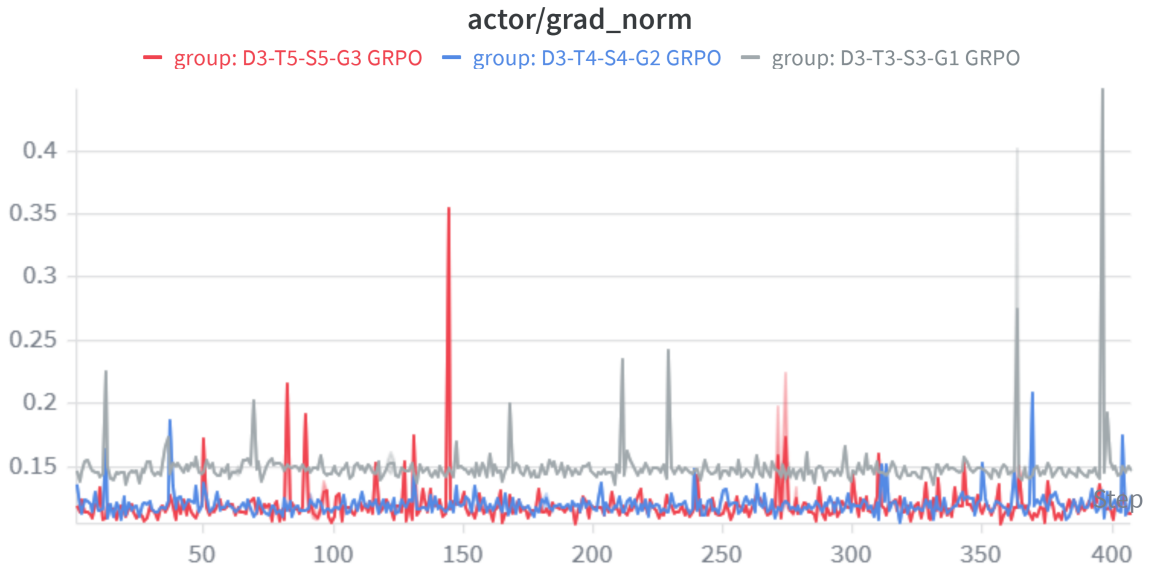


Figure 6: Actor gradient norm during GRPO training

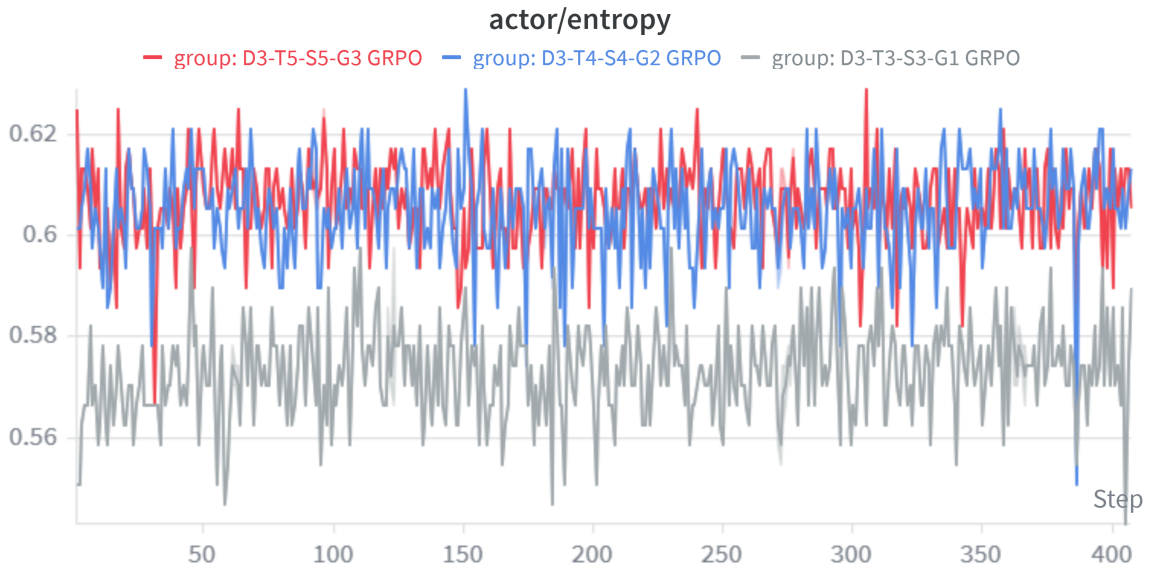


Figure 7: Actor entropy during GRPO training

6 Evaluation

Evaluation was conducted on a combined test set consisting of:

- **PrimeVul test set:** 407 vulnerability pairs (814 samples)
- **PrimeVul validation set:** 354 vulnerability pairs

- **SVEN validation set:** 209 vulnerability pairs (unused during training)

This yields a total of 970 pairs (1,940 samples). All results reported below use this combined test set unless otherwise stated. Most models were initially evaluated at multiple temperatures (0.2, 0.4, 0.6) to assess sensitivity but for our final tables we report results at a temperature of 0.6, which generally yielded the best P-C scores across all variants.

All models from the current iteration were evaluated using the training prompt format: a structured prompt with line numbers requesting JSON output, without any hints about vulnerability status or changed lines. Earlier iterations additionally used the `std_cls` format (a simple classification prompt requesting only YES/NO), which remains in the tables for historical comparison.

7 Results

7.1 Pair-wise Metrics

The pair-wise metrics measure the model's ability to correctly classify both samples in a vulnerability pair (vulnerable function + patched function):

- **P-C (Pair-Correct)** ↑: Both samples correctly classified
- **P-V (Pair-Vulnerable)** ↓: Both incorrectly labeled as vulnerable (false positive bias)
- **P-B (Pair-Benign)** ↓: Both incorrectly labeled as benign (false negative bias)
- **P-R (Pair-Reversed)** ↓: Labels swapped (vulnerable↔patched)

7.2 Combined Dataset Results

Table 1 presents evaluated models on the full combined test set. The table is split into evaluations using the newer "Training" prompt format (which tests the model's ability to output the complex JSON schema) and the older "std_cls" format (which allows comparison to earlier models).

For models evaluated across multiple runs, we report the peak metric alongside the average and maximum spread for each metric in the format: `Best (Avg ± Spread)`. Temperatures (T) are noted directly in the model names. Models are identified by a code `D#-T#-S#-G#` indicating their Dataset, Teacher Annotation, SFT, and GRPO versions respectively. Click any component of the code to jump to its methodology section.

Table 1: Combined Dataset Results – All Model Variants

Model	T	Acc ↑	F1 ↑	P-C (%) ↑	P-V (%) ↓	P-B (%) ↓	P-R (%) ↓	Parsable ↑
Training Prompt Format								
Baselines								
D3-T3-Base Qwen3-4B	0.6	52.57 (52.23 ± 0.67)	52.20 (51.92 ± 0.55)	20.68 (20.27 ± 0.83)	30.92 (31.13 ± 0.41)	32.91 (32.70 ± 0.41)	15.46 (15.88 ± 0.83)	1923/1940
D3-T3-Base Qwen3-32B (Teacher)	0.6	54.22 (54.47 ± 0.49)	59.31 (59.65 ± 0.68)	22.39 (22.39 ± 0.00)	44.47 (45.00 ± 1.04)	19.16 (19.21 ± 0.10)	13.95 (13.38 ± 1.14)	1927/1940
D2-T2-S2 SFT Qwen3-4B	0.6	54.87	58.23	25.51	37.47	20.57	16.43	1815/1876
D3-T3-S3 SFT Qwen3-4B	0.6	51.23 (50.40 ± 1.65)	41.71 (41.47 ± 0.48)	23.25 (22.30 ± 1.90)	11.96 (12.86 ± 1.81)	44.48 (43.66 ± 1.65)	20.29 (21.16 ± 1.74)	1696/1940
D3-T4-S4 SFT Qwen3-4B	0.6	54.45 (54.33 ± 0.48)	59.95 (59.89 ± 0.74)	23.12 (23.03 ± 0.19)	45.10 (44.93 ± 0.89)	17.88 (17.55 ± 0.54)	13.89 (14.47 ± 1.20)	1842/1940
D3-T5-S5 SFT Qwen3-4B	0.6	54.01 (53.58 ± 1.07)	57.92 (57.39 ± 1.23)	24.28 (23.05 ± 2.04)	39.18 (39.73 ± 1.59)	20.86 (21.65 ± 1.82)	15.67 (15.55 ± 2.40)	1870/1940
D3-T3-S3-G1 GRPO Qwen3-4B	0.6	56.11 (53.37 ± 4.15)	49.49 (46.84 ± 4.20)	28.38 (25.78 ± 4.81)	13.92 (14.60 ± 1.96)	40.56 (40.06 ± 2.65)	17.13 (19.54 ± 4.95)	1693/1940
D3-T4-S4-G2 GRPO Qwen3-4B	0.6	54.09 (53.62 ± 1.59)	58.11 (58.16 ± 1.08)	24.60 (22.82 ± 3.48)	39.14 (41.58 ± 4.17)	19.79 (19.88 ± 1.26)	16.44 (15.70 ± 1.37)	1856/1940
D3-T5-S5-G3 GRPO Qwen3-4B (step 180)	0.6	54.70	58.51	25.33	38.65	20.59	15.41	1872/1940
D3-T5-S5-G3 GRPO Qwen3-4B (step 407)	0.6	55.57 (54.18 ± 2.89)	58.37 (56.99 ± 2.72)	24.25 (23.68 ± 1.55)	38.09 (37.09 ± 2.12)	24.58 (23.81 ± 2.34)	13.06 (15.39 ± 4.21)	1875/1940
std_cls Prompt Format (Historical Comparison)								
D2-T2-Base Qwen3-4B	0.6	52.30	26.60	9.50	7.10	77.70	5.60	N/A
D2-T2-S2 SFT Qwen3-4B	0.6	55.01	53.93	26.46	26.03	30.78	16.74	N/A

7.3 PrimeVul Test Set Results

To establish a clear point of comparison against the original PrimeVul benchmarks and our early historical runs, Table 2 presents the results exclusively on the PrimeVul test split (407 pairs).

Table 2: PrimeVul Test Results – All Model Variants

Model	T	Acc ↑	F1 ↑	P-C (%) ↑	P-V (%) ↓	P-B (%) ↓	P-R (%) ↓	Parsable ↑
Training Prompt Format								
Baselines								
D3-T3-Base Qwen3-4B	0.6	51.66 (51.90 ± 0.48)	50.42 (50.39 ± 0.05)	19.80 (19.73 ± 0.14)	28.87 (28.94 ± 0.13)	34.36 (35.40 ± 2.08)	16.94 (15.90 ± 2.07)	842/848
D3-T3-Base Qwen3-32B (Teacher)	0.6	54.03 (53.53 ± 1.01)	58.69 (58.37 ± 0.64)	21.19 (20.59 ± 1.19)	44.28 (44.76 ± 0.95)	21.66 (21.19 ± 0.95)	12.85 (13.45 ± 1.19)	842/848
D2-T2-S2 SFT Qwen3-4B	0.6	56.03	58.90	26.17	36.12	23.03	14.65	787/814
D3-T3-S3 SFT Qwen3-4B	0.6	50.47 (49.96 ± 1.02)	39.27 (39.70 ± 0.85)	23.17 (21.45 ± 3.42)	10.06 (11.92 ± 3.73)	46.34 (45.74 ± 1.20)	20.42 (20.87 ± 0.88)	743/848
D3-T4-S4 SFT Qwen3-4B	0.6	54.48 (53.75 ± 1.95)	59.16 (58.87 ± 0.80)	23.97 (22.41 ± 4.54)	42.60 (44.01 ± 3.77)	18.62 (18.92 ± 0.54)	14.79 (14.64 ± 0.92)	813/848
D3-T5-S5 SFT Qwen3-4B	0.6	53.83 (53.29 ± 1.26)	56.18 (56.20 ± 1.27)	23.94 (22.15 ± 3.73)	34.91 (37.92 ± 4.73)	25.18 (24.51 ± 2.13)	15.96 (15.39 ± 1.06)	821/848
D3-T3-S3-G1 GRPO Qwen3-4B	0.6	55.44 (51.34 ± 6.87)	48.01 (43.88 ± 6.96)	25.93 (22.74 ± 6.39)	15.31 (14.88 ± 2.26)	44.06 (42.42 ± 8.74)	14.68 (19.94 ± 11.77)	734/848
D3-T4-S4-G2 GRPO Qwen3-4B	0.6	53.50 (51.44 ± 3.32)	57.52 (55.72 ± 3.32)	23.40 (20.46 ± 6.11)	39.44 (40.65 ± 3.83)	20.61 (21.46 ± 5.39)	16.53 (17.41 ± 2.89)	813/848
D3-T5-S5-G3 GRPO Qwen3-4B (step 180)	0.6	52.99	56.85	23.03	38.98	21.51	16.45	817/848
D3-T5-S5-G3 GRPO Qwen3-4B (step 407)	0.6	53.97 (52.97 ± 4.05)	55.86 (55.28 ± 3.86)	23.23 (21.99 ± 2.97)	34.84 (36.26 ± 4.44)	26.01 (25.45 ± 4.62)	15.90 (16.29 ± 4.58)	817/848
std_cls Prompt Format (Historical Comparison)								
Early Iterations (816 samples)								
PrimeVul GPT-4 (Reported)	-	-	-	12.94	54.26	24.47	8.33	N/A
D1-T1-Base Qwen3-8B	0.6	52.80	32.50	12.78	9.09	70.02	8.11	N/A
D1-T1-S1 Qwen3-8B (LoRA)	0.6	52.80	42.70	18.67	14.74	52.58	14.00	N/A
D2-T2-Base Qwen3-4B	0.6	50.30	18.90	6.20	5.40	83.00	5.40	N/A
D2-T2-S2 SFT Qwen3-4B	0.6	55.41	53.28	26.29	24.57	33.66	15.48	N/A

7.4 Analysis

The results reveal a clear progression in discriminative performance across iterations of our pipeline, with the Pair-wise Correct (P-C) metric serving as the primary indicator of improvement.

7.4.1 Overall Progression

Our best model, the GRPO Run 1 variant (D3-T3-S3-G1), achieves a peak P-C of **28.38%** on the combined test set and **25.93%** on the PrimeVul test split, the latter representing a 100% relative improvement over the GPT-4 result reported in the original PrimeVul paper (12.94% on the same split) and a 39% relative improvement over our own initial LoRA fine-tuning baseline on Qwen3-8B (18.67%). Crucially, this GRPO model also surpasses all SFT-only variants, the best of which (D2-T2-S2) reaches 25.51% P-C on the combined set, demonstrating that the reinforcement learning stage provides a measurable gain beyond what supervised fine-tuning alone can achieve.

7.4.2 Initial Experiments: LoRA Fine-Tuning on Qwen3-8B

In our first iteration (D1-T1-S1), we tested the viability of reasoning-augmented fine-tuning using Low-Rank Adaptation (LoRA) on a 4-bit quantized Qwen3-8B model, trained strictly on the PrimeVul paired dataset. The fine-tuned model demonstrated a significant relative improvement over its stock baseline on the PrimeVul test set: the F1-score increased from 0.325 to 0.427, and the crucial Pair-wise Correct (P-C) metric rose from 12.78% to 18.67% (an approximately 46% relative increase). This improvement primarily came from a reduction in the model's tendency to falsely label both samples as benign, proving that explicitly teaching the model to reason about vulnerabilities was a promising direction for scaling up to full parameter fine-tuning and reinforcement learning.

7.4.3 SFT Results

Among the fully finetuned (Qwen3-4B) SFT-only models, the earliest full-parameter checkpoint (D2-T2-S2) achieves 25.51% P-C on the combined set, already surpassing all earlier approaches. The later SFT variants on the function-level deduplicated dataset show an interesting split: the hint-free SFT (D3-T3-S3) achieves a competitive 23.25% P-C but at the cost of significantly higher P-B (44.48%) and lower parsability (1,696/1,940), suggesting that using teacher responses generated using hints then removing all metadata context from the training prompts made the model more conservative and more likely to not finish reasoning before reaching the maximum token limit of the model. The reintroduction of hints into the SFT training prompts ablation (D3-T4-S4) partially addresses both issues, achieving 23.12% P-C with stronger parsability (1,842/1,940) and the highest F1 among SFT models (59.95%), though it exhibits high P-V (45.10%), indicating the model overpredicts vulnerabilities. The final SFT variant with refined teacher annotations (D3-T5-S5) strikes a middle ground with 24.28% P-C, more balanced P-V/P-B distribution, and strong parsability (1,870/1,940).

7.4.4 GRPO Results

GRPO training amplifies the SFT model's discriminative capability, but the magnitude of improvement depends heavily on the SFT base and reward configuration. Run 1 (D3-T3-S3-G1), starting from the hint-free SFT base with the original 4-tier rewards, achieves the highest P-C (28.38%) of any model in our experiments, exceeding its SFT base by over 5 percentage points. However, it also exhibits the highest variance across evaluation temperatures (spread of $\pm$4.81), the lowest parsability (1,693/1,940), and an elevated P-B (40.56%). This suggests that while the hint-free pipeline produces a model with the strongest discrimination signal, it does so at the cost of output stability.

Run 2 (D3-T4-S4-G2), using the 5-tier rewards and hints-in-SFT base, shows more modest P-C improvement over its SFT checkpoint (24.60% vs 23.12%) but maintains higher parsability and lower variance. The 5-tier structure's additional reward granularity did not translate to better discrimination; instead, analysis of training rollouts revealed it incentivised the model to "spray" large numbers of predicted lines to hit the 75% overlap threshold for the 1.25 reward tier.

Run 3 (D3-T5-S5-G3), with the adjusted 4-tier rewards and data quality guards, was evaluated at two checkpoints. The step 180 checkpoint (25.33% P-C) slightly outperforms its SFT base (24.28%), while the step 407 checkpoint (24.25% P-C) shows no improvement, potentially indicating that extended training without the aggressive hint-free starting point leads to diminishing returns. However, both checkpoints maintain excellent parsability ($\sim$1,872/1,940) and the most balanced P-V/P-B trade-off among GRPO models.

7.4.5 Other Observations

The teacher model (Qwen3-32B) achieves 22.39% P-C on the combined set and 21.19% on PrimeVul, confirming that the student model's fine-tuned variants genuinely surpass the teacher's zero-shot capability. Interestingly, the SFT prompt format shapes the student's default behaviour at evaluation time (when no hints are present): all hints-in-SFT models, trained with explicit vulnerability cues such as CVE descriptions and changed line numbers, become reliant on those cues and over-predict vulnerabilities when they are absent (elevated P-V), while the hint-free SFT model, never exposed to metadata context during training, becomes conservative and defaults to benign predictions (high P-B). The GRPO stage modulates these biases further, with hint-free GRPO models showing the most favourable P-C/P-B trade-off.

Across the historical comparison rows (std_cls prompt), the pattern is consistent: base models struggle significantly (Qwen3-4B achieves only 6.20% P-C with std_cls, defaulting to labelling everything as benign), while SFT models perform comparably regardless of prompt format ($\sim$26% P-C), demonstrating that fine-tuning teaches the model to engage with the vulnerability detection task regardless of prompt style.

7.5 Examples

To illustrate the practical impact of fine-tuning, we present examples where the SFT model correctly detected vulnerabilities that the base model missed:

Example 1: TensorFlow – Missing Parameter Validation (CWE-20)

```
Status SetUnknownShape(const NodeDef* node, int output_port) {
    shape_inference::ShapeHandle shape =
        GetUnknownOutputShape(node, output_port);

    InferenceContext* ctx = GetContext(node);
    if (ctx == nullptr) {
        return errors::InvalidArgument("Missing context");
    }

    ctx->set_output(output_port, shape);  // output_port NOT validated!
    return Status::OK();
}
```

Listing 6: TensorFlow SetUnknownShape function with missing validation

The vulnerability occurs because `output_port` is validated in `GetUnknownOutputShape` but not before being passed to `ctx->set_output()`, which could lead to out-of-bounds access. The base model predicted this as **NOT VULNERABLE**, while the SFT model correctly identified it as **VULNERABLE**. This is reflected in their reasoning:

- Base: "...No user input, internal API, ctx null check exists..."
- SFT: "...output_port could be out-of-bounds, no validation before set_output..."

Example 2: deark – Division by Zero from File Input (CWE-369)

```
void fmtutil_macbitmap_read_pixmap_only_fields(deark *c,
        dbuf *f, struct fmtutil_macbitmap_info *bi, i64 pos) {

    bi->pixelsize = dbuf_getu16be(f, pos+18);  // Read from file

    // No validation of pixelsize!
    bi->pdwidth = (bi->rowbytes * 8) / bi->pixelsize;  // Division by
   zero!

    if (bi->pdwidth < bi->npwidth) {
        bi->pdwidth = bi->npwidth;
    }
}
```

Listing 7: deark bitmap parsing with division by zero vulnerability

Here, `pixelsize` is read directly from user-controlled file input without validation. If the file contains 0 for this field, it causes a division by zero crash. The base model missed

this vulnerability, assuming valid data and predicting **NOT VULNERABLE**, while the SFT model treats file input as untrusted, and correctly detected it as **VULNERABLE**. Again, this is reflected in the reasoning:

- Base: "...Processing known file format, data expected to be valid..."
- SFT: "...pixelsize read from untrusted input, division by zero if zero..."

Example 3: TensorFlow – Unchecked Iterator Access (CWE-824)

```
Status GetInitOp(...) {
  const auto& init_op_sig_it =
      meta_graph_def.signature_def().find(kSavedModelInitOpSignatureKey
    );
  if (init_op_sig_it != sig_def_map.end()) {
    *init_op_name = init_op_sig_it->second.outputs()
                        .find(kSavedModelInitOpSignatureKey)
                        ->second.name();  // Potential crash!
  }
  return Status::OK();
}
```

Listing 8: TensorFlow GetInitOp with unchecked iterator dereference

The vulnerability occurs when accessing `init_op_sig_it->...->second.name()`. If the inner `find()` returns `end()`, dereferencing it causes undefined behavior (crash or memory corruption). The SFT model's reasoning:

```
Another angle: Are there any unchecked assumptions? When accessing
   init_op_sig_it->second.outputs().find(...)}, if the outputs don't
   have the key, find() would return an iterator to end, and then
   accessing .name() would be undefined behavior... This is a possible
   use-after-free or crash, which could be exploited.
```

Example 4: GPAC – Out-of-Bounds Array Access (CWE-125)

```
static s32 svc_parse_slice(GF_BitStream *bs, AVCState *avc,
                           AVCSliceInfo *si) {
    s32 pps_id = gf_bs_read_ue_log(bs, "pps_id");
    if (pps_id > 255) return -1;

    si->pps = &avc->pps[pps_id];            // Checked OK
    si->pps->id = pps_id;

    si->sps = &avc->sps[si->pps->sps_id + GF_SVC_SSPS_ID_SHIFT];  //
   NOT CHECKED!
    if (!si->sps->log2_max_frame_num) return -2;
    // ...
}
```

Listing 9: GPAC SVC slice parsing with missing bounds check

The code validates pps_id before using it as an array index, but fails to validate sps_id + GF_SVC_SSPS_ID_SHIFT before accessing the sps array. If sps_id is large, this causes out-of-bounds memory access. The base model missed this because it saw the pps_id validation and assumed all indices were checked. The SFT model correctly identified the missing validation on the second array access.

Example 5: TensorFlow – Memory Exhaustion (CWE-400) – False Negative

This example shows a case where *both* models failed:

```
Status ConstantFolding::IsSimplifiableReshape(const NodeDef& node,
                              const GraphProperties& properties) const {
  const NodeDef* new_shape = node_map_->GetNode(node.input(1));
  if (!IsReallyConstant(*new_shape)) {
    return errors::InvalidArgument("Shape must be constant");
  }

  TensorVector outputs;
  Status s = EvaluateNode(*new_shape, TensorVector(), &outputs);

  if (outputs[0]->dtype() == DT_INT32) {
    std::vector<int32> shp;
    for (int i = 0; i < outputs[0]->NumElements(); ++i) {  // No bounds
    check!
      int32_t dim = outputs[0]->flat<int32>()(i);
      shp.push_back(dim);  // Memory exhaustion if NumElements() is
   huge
    }
  }
}
```

Listing 10: TensorFlow constant folding with unbounded allocation

The vulnerability: NumElements() comes from user-controlled input (the tensor shape). An attacker can craft a tensor claiming to have 2^{31} elements, causing the push_back operations to exhaust memory (Denial of Service). Both models were distracted by the error handling and missed the missing bounds check on the loop iteration count.

These examples demonstrate that fine-tuning helps the model recognize vulnerability patterns including: parameter validation requirements across function calls, security implications of untrusted input in arithmetic operations, unchecked iterator dereferences, and missing bounds checks on array indices. However, other vulnerabilities like resource exhaustion through unbounded loops remain challenging.

8 Conclusion and Future Work

This work demonstrates that reasoning-augmented fine-tuning, combined with reinforcement learning, substantially improves the ability of LLMs to distinguish between vulnerable code and its patched counterpart. We developed a multi-stage pipeline consisting of dataset consolidation, teacher-guided annotation, supervised fine-tuning, and Group Relative Policy Optimization (GRPO), evaluated across multiple configurations on a combined benchmark of 970 vulnerability pairs.

Our consolidated dataset pipeline merged three complementary vulnerability sources (PrimeVul, SVEN, and SecVulEval) into 12,912 unique deduplicated functions through a function-level deduplication strategy that recovered thousands of pairs previously lost by commit-level collapsing. The Qwen3-32B teacher model generated reasoning-augmented training annotations, enabling the student model to learn not just the classification outcome but the analytical process leading to it.

The best-performing model, a GRPO-trained Qwen3-4B using the 4-tier reward structure with hint-free prompts, achieved a Pair-wise Correct score of 28.38% on the combined test set and 25.93% on the PrimeVul test split, the latter representing a 100% relative improvement over GPT-4's reported performance on PrimeVul (12.94%) and a 39% improvement over our initial LoRA baseline (18.67%). This GRPO model also surpasses all SFT-only variants, the best of which reaches 25.51% P-C, confirming that reinforcement learning provides a measurable gain beyond supervised fine-tuning alone. Our experiments also revealed important design trade-offs: hint-free SFT training produces models with stronger discrimination but lower output parsability, while hints-in-SFT models exhibit better format compliance but inherit the teacher's vulnerability-prediction bias. The GRPO stage consistently improves P-C over SFT baselines, though run-to-run variance remains notable, and iterating on the reward structure proved important for controlling undesirable behaviours such as line prediction spraying.

However, all configurations still show significant inconsistencies, like high evaluation variance across temperatures, sensitivity to reward structure design, and a trade-off between output parsability and discrimination capability. The overall P-C scores, while substantially improved over baselines, are still too low for reliable real-world deployment, underscoring that considerable room for improvement exists in both training methodology and evaluation robustness.

Several directions remain for future work:

- **Continuous reward functions:** replacing the discrete reward tiers with a smooth, percentage-based reward scaled by line localization accuracy could provide finer gradient signals during GRPO training.
- **Newer foundation models:** code-specialised models with stronger inherent understanding of security patterns may yield better starting points for the fine-tuning pipeline.
- **SAST-assisted rewards:** integrating static analysis tool outputs into the reward function could provide more semantically grounded feedback than purely overlap-based line

matching.

- **Repository-level context:** extending the approach beyond isolated functions to incorporate call graphs and data-flow information, as suggested by recent benchmarks like CyberGym [5] and JitVul [6], would better reflect real-world vulnerability detection scenarios.

Resources

Models and datasets from this research are publicly available:

- Main code repository
- SFT Model
- GRPO Model
- Merged Dataset
- Fine-tuning Dataset

References

[1] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. Vulnerability detection with code language models: How far are we?, 2024.

[2] Arastoo Zibaeirad and Marco Vieira. Vulnllmeval: A framework for evaluating large language models in software vulnerability detection and patching, 2024.

[3] Yue Li, Xiao Li, Hao Wu, Yue Zhang, Xiuzhen Cheng, Yating Liu, Fengyuan Xu, and Sheng Zhong. If llms would just look: Simple line-by-line checking improves vulnerability localization, 2025.

[4] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag, 2024.

[5] Zhun Wang, Tianneng Shi, Jingxuan He, Matthew Cai, Jialin Zhang, and Dawn Song. Cybergym: Evaluating ai agents' real-world cybersecurity capabilities at scale, 2025.

[6] Alperen Yildiz, Sin G. Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil M. Divakaran. Benchmarking llms and llm-based agents in practical vulnerability detection for code repositories, 2025.

[7] Radowanul Haque, Aftab Ali, Sally McClean, and Naveed Khan. Explainable vulnerability detection in c/c++ using edge-aware graph attention networks, 2025.

[8] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, 2019.

[9] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. Vuldeepecker: A deep learning-based system for vulnerability detection. In *Proceedings 2018 Network and Distributed System Security Symposium*, NDSS 2018. Internet Society, 2018.

[10] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing*, 19(4):2244–2258, July 2022.

[11] Benjamin Steenhoek, Hongyang Gao, and Wei Le. Dataflow analysis-inspired deep learning for efficient vulnerability detection, 2023.

[12] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024.

[13] Daniel Han, Michael Han, and Unsloth team. Unsloth, 2023.

[14] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020.

[15] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.

[16] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Kashif Sharma, Shengyi Huang, Rishabh Mishra, Nathan Manber, et al. Trl: Transformer reinforcement learning, 2020.

[17] Qiying Yu, Zheng Zhang, Ruofei Wan, Xin Xing, and Baichun Wu. Dapo: An open-source llm reinforcement learning system at scale, 2025.

[18] Yiping Liu, Zijian Li, Michael Hall, Percy Liang, and Tatsunori Hashimoto. Understanding r1-zero-like training: A critical perspective, 2025.